

数据库系统

Ch11 —数据库恢复

详细学习笔记

Database Systems: Database Recovery
Comprehensive Study Notes

中英双语对照 | Page-Cited Key Points

分类: 数据库系统
日期: 2026 年 6 月 21 日

目录

1	10.1 事务的基本概念 (Transaction Basic Concepts)	2
1.1	一、什么是事务 (What is a Transaction)	2
1.2	二、如何定义事务 (How to Define a Transaction)	2
1.3	三、事务的 ACID 特性 (ACID Properties)	2
2	10.2 数据库恢复概述 (Recovery Overview)	4
3	10.3 故障的种类 (Types of Failures)	4
3.1	一、事务故障 (Transaction Failure)	4
3.2	二、系统故障 (System Failure / Soft Crash)	4
3.3	三、介质故障 (Media Failure / Hard Crash)	4
4	恢复操作的基本原理	5
5	10.4 恢复的实现技术 (Recovery Implementation Techniques)	6
5.1	10.4.1 数据转储 (Data Dump / Backup)	6
5.2	10.4.2 登记日志文件 (Logging)	8
6	10.5 恢复策略 (Recovery Strategies)	10
6.1	10.5.1 事务故障的恢复	10
6.2	10.5.2 系统故障的恢复	10
6.3	10.5.3 介质故障的恢复	11
7	10.6 具有检查点的恢复技术 (Checkpoint-based Recovery)	12
7.1	一、动机 (Motivation)	12
7.2	二、检查点技术 (Checkpoint Technique)	12
7.3	三、利用检查点的恢复策略	13
8	10.7 数据库镜像 (Database Mirroring)	14
9	小结 (Summary)	14
	核心考点速查表	15

10.1 事务的基本概念 (Transaction Basic Concepts)

一、什么是事务 (What is a Transaction)

- **事务 (Transaction)**是用户定义的一个数据库操作序列，这些操作要么全做，要么全不做，是一个不可分割的工作单位。

Transaction: a user-defined sequence of database operations; either all or none —an indivisible unit of work. [p.5]

- 事务和程序是两个概念。一个应用程序通常包含多个事务；在关系数据库中，一个事务可以是一条 SQL 语句、一组 SQL 语句或整个程序。

A program may contain multiple transactions; a transaction can be one SQL statement, a group of SQL statements, or an entire program. [p.5]

- 事务是恢复和并发控制的基本单位。

Transaction is the basic unit of both recovery and concurrency control. [p.5]

二、如何定义事务 (How to Define a Transaction)

- **显式定义 (Explicit):**

BEGIN TRANSACTION

SQL 语句 1

SQL 语句 2

...

COMMIT 或 ROLLBACK

[p.6]

- **隐式定义 (Implicit):** 当用户没有显式定义事务时，DBMS 按缺省规定自动划分事务。
[p.6]

- **COMMIT:** 事务正常结束，提交所有操作，对数据库的更新永久生效。

Normal termination; all updates become permanent. [p.7]

- **ROLLBACK:** 事务异常终止，回滚所有更新操作，事务滚回到开始时的状态。

Abnormal termination; all updates rolled back to initial state. [p.7]

三、事务的 ACID 特性 (ACID Properties)

- **一致性状态 (Consistent State):** 数据库中只包含成功事务提交的结果。 [p.10]
- **不一致状态 (Inconsistent State):** 数据库中包含失败事务的结果。 [p.10]

表 1: ACID 特性总结

字母	中文	English	简要说明
A	原子性	Atomicity	事务中诸操作要么全做，要么全不做 [p.9]
C	一致性	Consistency	事务执行结果使数据库从一致性状态变到另一一致性状态 [p.10]
I	隔离性	Isolation	一个事务的执行不能被其他事务干扰 [p.12]
D	持续性	Durability	事务一旦提交，改变就应是永久性的 [p.13]

- 银行转账例子: 从 A 取钱存入 B。事务 $A = A - 1$ 和 $B = B + 1$ 必须全做或全不做，否则数据库不一致。 [p.11]
- 隔离性问题示例: T1 和 T2 并发。T1 读 $A=16$ ， $A=A-1$ 写回 15；T2 读 $A=16$ （旧值）， $A=A-3$ 写回 13。T1 的修改被 T2 覆盖（丢失更新 **Lost Update**）。 [p.12]
- 破坏 ACID 的因素: (1) 并发事务交叉执行破坏隔离性；(2) 事务被强行停止破坏原子性/持续性。 [p.14]

10.2 数据库恢复概述 (Recovery Overview)

- **故障不可避免:** 计算机硬件故障、系统软件/应用软件错误、操作员失误、恶意破坏。
[p.16]
- **故障影响:** 运行事务非正常中断, 破坏数据库。 [p.16]
- **恢复子系统:** DBMS 提供恢复子系统, 保证故障后能把数据库从错误状态恢复到逻辑一致状态。恢复子系统的代码占全部代码的 10% 以上。 [p.17]

10.3 故障的种类 (Types of Failures)

一、事务故障 (Transaction Failure)

- 事务在运行过程中未运行至正常终止点就夭折。 [p.18]
- **常见原因:** 输入数据有误、运算溢出、违反完整性限制、应用程序出错、并行事务死锁。 [p.18]
- **恢复方法:** **UNDO/ROLLBACK** —清除该事务对数据库的所有修改, 使其像根本没有启动过一样。 [p.19]

二、系统故障 (System Failure / Soft Crash)

- 整个系统正常运行突然被破坏, 所有正在运行的事务非正常终止, 内存缓冲区信息全部丢失, 但外存数据未受影响。 [p.20]
- **常见原因:** OS/DBMS 代码错误、操作员失误、CPU 故障、突然停电。 [p.21]
- **恢复方法:** 系统重启时:
UNDO —强行撤消所有未完成事务 (可能已写入磁盘)
REDO —重做所有已提交的事务 (可能还在缓冲区未写入磁盘) [p.22]

三、介质故障 (Media Failure / Hard Crash)

- 硬件故障使外存数据部分或全部丢失。**可能性小但破坏性大。** [p.24]
- **常见原因:** 磁盘损坏、磁头碰撞、OS 潜在错误、强磁场干扰。 [p.25]
- **恢复方法:** 装入故障前某时刻的数据副本, REDO 所有成功事务。 [p.26]

表 2: 三种故障类型对比

故障类型	影响范围	恢复方法	是否需要 DBA
事务故障	单个事务	UNDO	自动, 不需要
系统故障	所有运行事务、内存缓冲区	UNDO+REDO	自动, 不需要
介质故障	外存数据	重装备份 +REDO	需要 DBA 介入

恢复操作的基本原理

- 关键原理: 冗余 (Redundancy)

利用存储在系统其它地方的冗余数据来重建数据库中已被破坏或不正确的那部分数据。

Use redundant data stored elsewhere to reconstruct damaged or incorrect portions of the database.

[p.27]

10.4 恢复的实现技术 (Recovery Implementation Techniques)

两个关键问题:

1. 如何建立冗余数据: 数据转储 (Backup) + 登录日志文件 (Logging)
2. 如何利用冗余数据实施恢复

[p.28]

10.4.1 数据转储 (Data Dump / Backup)

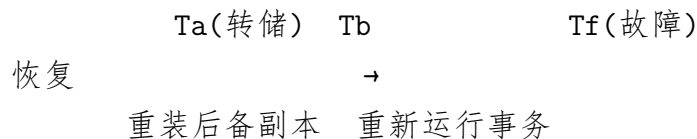
一、什么是转储

- DBA 将整个数据库复制到磁带或另一个磁盘上保存。备用的数据文本称为**后备副本**(Backup Copy)。

[p.30]

恢复过程示意:

正常运行



二、转储方法分类

1. 静态转储 vs 动态转储 (Static vs Dynamic)

2. 海量转储 vs 增量转储 (Full vs Incremental)

[p.38]

- 海量转储 (Full dump): 每次转储全部数据库。恢复更方便。
- 增量转储 (Incremental dump): 只转储上次转储后更新过的数据。数据库大且事务频繁时更实用。

3. 四种组合

三、转储策略实例

[p.40]

- 每天晚上: 动态增量转储
- 每周一次: 动态海量转储
- 每月一次: 静态海量转储

	静态转储 (Static)	动态转储 (Dynamic)
条件	系统中无运行事务时转储 [p.32]	与用户事务并发进行 [p.34]
一致性	副本保证一致性	副本不能保证正确有效
可用性	转储期间不允许存取/修改, 可用性降低	不影响事务运行
恢复方式	只需重装备份即可恢复	需后备副本 + 日志文件
优点	实现简单	不等用户事务结束
缺点	事务必须等转储, 新事务等转储结束	必须配合日志文件

	静态转储	动态转储
海量转储	静态海量转储	动态海量转储
增量转储	静态增量转储	动态增量转储

表 3: 转储方法的四种组合

10.4.2 登记日志文件 (Logging)

一、日志文件的内容

- **日志文件 (Log):** 记录事务对数据库的更新操作的文件。 [p.43]
- **两种格式:** 基于记录的日志文件 (Record-based) / 基于数据块的日志文件 (Block-based)
- **日志应包含:**
 - 各事务的开始标记 (BEGIN TRANSACTION)
 - 各事务的结束标记 (COMMIT / ROLLBACK)
 - 各事务的所有更新操作
 - 与事务有关的内部更新操作

[p.44]

基于记录的日志文件 (Record-based Log)

[p.45]

每条日志记录包含:

1. 事务标识 (Transaction ID)
2. 操作类型 (Operation Type): 插入/删除/修改
3. 操作对象 (Target): 记录 ID、Block No.
4. **更新前旧值 BI** (Before Image): 插入操作此项为空
5. **更新后新值 AI** (After Image): 删除操作此项为空

基于数据块的日志文件 (Block-based Log)

[p.46]

- 更新前数据所在的整个数据块的值（插入操作为空）
- 更新后整个数据块的值（删除操作为空）

二、日志文件的用途

[p.47-50]

1. 进行事务故障恢复
2. 进行系统故障恢复
3. 协助后备副本进行介质故障恢复

静态转储 + 日志: 转储后仍定期转储日志; 故障时重装备份, 利用日志 REDO 已完成事务。[p.48] **动态转储 + 日志**: 动态转储同时转储同一时点日志; 两者结合才能恢复一致性状态。[p.50]

三、登记日志文件的原则

[p.51-52]

1. **时间次序原则**: 登记的次序严格按并行事务执行的时间次序。[p.51]
2. **先写日志原则 (Write-Ahead Logging / WAL)**: 必须先写日志文件, 后写数据库。[p.51]

为什么先写日志?

[p.52]

- 如果先写了数据库修改而日志未登记 → 故障后无法恢复
- 如果先写日志但数据库未修改 → 恢复时最多多执行一次不必要的 UNDO, 不影响正确性

10.5 恢复策略 (Recovery Strategies)

10.5.1 事务故障的恢复

[p.54-56]

- 事务在运行至正常终止点前被中止。恢复由系统自动完成，无需用户干预。 [p.54]

恢复步骤 (UNDO 过程):

1. 反向扫描日志文件 (从最后向前), 查找该事务的更新操作。 [p.55]
2. 对更新操作执行逆操作——将 BI 写入数据库: [p.55]
 - 插入操作: BI 为空 → 执行删除操作
 - 删除操作: AI 为空 → 执行插入操作
 - 修改操作: 用 BI 代替 AI
3. 继续反向扫描, 处理其他更新操作。 [p.56]
4. 直至读到该事务的 **BEGIN TRANSACTION** 标记, 恢复完成。 [p.56]

10.5.2 系统故障的恢复

[p.57-59]

- 造成不一致的原因:
 - 未完成事务的更新已写入物理数据库
 - 已提交事务的更新还留在内存缓冲区未写入数据库
- [p.57]
- 恢复方法 = **UNDO** + **REDO**: UNDO 未完成事务, REDO 已提交事务。系统重启时自动完成。 [p.57]

恢复步骤:

1. 正向扫描日志文件, 找出两个队列: [p.58]
 - Redo 队列: 故障发生前已提交的事务 (如 T1, T3, T8...)
 - Undo 队列: 故障发生时未完成的事务 (如 T2, T4, T5, T6, T7, T9...)

2. 对 **Undo** 队列执行 **UNDO**: 反向扫描日志, 对每个 UNDO 事务的更新操作执行逆操作。 [p.59]
3. 对 **Redo** 队列执行 **REDO**: 正向扫描日志, 对每个 REDO 事务重新执行登记的操作。 [p.59]

10.5.3 介质故障的恢复

[p.60-65]

恢复方法:

1. 重装数据库, 使数据库恢复到一致性状态
2. 重做已完成的事务 (REDO)

[p.60-61]

恢复步骤:

1. 装入最新的后备数据库副本:
 - **静态转储副本**: 装入后即处于一致性状态
 - **动态转储副本**: 还须同时装入转储时刻的日志文件, 利用 REDO+UNDO 恢复到一致性状态
 2. 装入日志文件副本, **重做已完成事务**: 扫描日志找出已提交事务 ID 记入重做队列, 正向扫描执行 REDO (将 AI 写入数据库)。 [p.61]
 3. 介质故障恢复需要 **DBA 介入**: DBA 重装备份和日志, 执行恢复命令; 具体操作由 DBMS 完成。 [p.64]
- [p.65]

10.6 具有检查点的恢复技术 (Checkpoint-based Recovery)

一、动机 (Motivation)

两个问题:

[p.67]

1. 搜索整个日志文件耗费大量时间
2. REDO 处理会重新执行已写入数据库的操作, 浪费大量时间

二、检查点技术 (Checkpoint Technique)

[p.68-73]

解决方案:

- 在日志文件中增加**检查点记录**(checkpoint record)
- 增加**重新开始文件**(restart file)
- 恢复子系统在登录日志期间动态维护日志

[p.68]

检查点记录的内容:

[p.70]

1. 建立检查点时刻所有正在执行的事务清单 (Active list)
2. 这些事务最近一个日志记录的地址

重新开始文件的内容: 记录各个检查点记录在日志文件中的地址。

[p.70]

维护日志文件的 4 个步骤:

[p.72]

1. 将当前日志缓冲区所有日志记录写入磁盘日志文件
2. 在日志文件中写入一个检查点记录
3. 将当前数据缓冲区所有数据记录写入磁盘数据库
4. 把检查点记录在日志文件中的地址写入重新开始文件

建立检查点的时机:

[p.73]

- **定期:** 按预定时间间隔
- **不定期:** 按某种规则 (如日志文件已写满一半)

三、利用检查点的恢复策略

[p.74-78]

- 事务 T 在检查点之前提交: 其修改已写入数据库, 无需 REDO。 [p.74]

关键图示 (T_c = 检查点时刻, T_f = 系统故障时刻): [p.75]

事务	时间关系	处理方式
T1	在 T_c 前提交	不要 REDO
T2	跨 T_c , T_c 后提交	REDO
T3	T_c 后开始, T_f 时未提交	UNDO
T4	T_c 后开始并提交	REDO
T5	T_c 后开始, T_f 时未提交	UNDO

表 4: 检查点恢复中五种事务的处理

利用检查点的具体恢复步骤: [p.76-78]

- 从重新开始文件中找到最后一个检查点记录在日志中的地址。 [p.76]
- 由该地址在日志文件中找到最后一个检查点记录。 [p.76]
- 从检查点记录得到 ACTIVE-LIST (检查点时刻所有正在执行的事务清单)。 [p.77]
建立两个队列:
 - UNDO-LIST: 初始放入 ACTIVE-LIST 中的所有事务
 - REDO-LIST: 初始为空
- 从检查点开始正向扫描日志文件直至文件结束: [p.78]
 - 遇到新开始的事务 $T_i \rightarrow$ 放入 UNDO-LIST
 - 遇到提交的事务 $T_j \rightarrow$ 从 UNDO-LIST 移到 REDO-LIST
- 对 UNDO-LIST 中每个事务执行 UNDO, 对 REDO-LIST 中每个事务执行 REDO。 [p.78]

10.7 数据库镜像 (Database Mirroring)

- 介质故障影响最严重，恢复费时。提高可用性的方案: **数据库镜像 (Mirror)**。 [p.79]
- **什么是镜像**: DBMS 自动把整个数据库或关键数据复制到另一个磁盘，自动保证镜像数据与主数据的一致性。 [p.80]
- **镜像的双重用途**: [p.81]
 1. **出现介质故障时**: 自动利用镜像磁盘数据恢复，**不需要关闭系统和重装数据库副本**
 2. **没有故障时**: 可用于并发操作——一个用户对主数据加排他锁修改时，其他用户可以读镜像数据

小结 (Summary)

- 数据库一致性状态: 只包含成功事务提交的结果。 [p.83]
- DBMS 保证事务的 **ACID** 特性: 原子性、一致性、隔离性、持续性。 [p.83]
- DBMS 必须对**三类故障**进行恢复: 事务故障、系统故障、介质故障。 [p.84]
- 最常用的恢复技术: 数据库转储 (Backup) + 登记日志文件 (Logging)。 [p.84]
- 恢复基本原理: 利用存储在后备副本、日志文件和数据库镜像中的**冗余数据**重建数据库。 [p.84]

表 5: 常用恢复方法总结

故障类型	恢复方法
事务故障 (Transaction Failure)	UNDO
系统故障 (System Failure)	UNDO + REDO
介质故障 (Media Failure)	重装备份恢复一致性 + REDO

表 6: 提高恢复效率的技术

技术	作用
检查点技术 (Checkpoint)	提高系统故障恢复效率；提高动态转储介质故障恢复效率
镜像技术 (Mirroring)	改善介质故障恢复效率，可实现即时切换

核心考点速查表 (Key Exam Points Quick Reference)

1. 事务定义 + 事务是恢复和并发控制的基本单位 [p.5]
2. ACID 四特性: Atomicity, Consistency, Isolation, Durability [p.8-13]
3. 三种故障类型: 事务故障 / 系统故障 / 介质故障 [p.18-26]
4. 恢复基本原理: 冗余 (Redundancy) [p.27]
5. 转储分类: 静态/动态 × 海量/增量—四种组合 [p.32-39]
6. 日志记录内容: TID + 操作类型 + 操作对象 + BI + AI [p.45]
7. 先写日志原则 (WAL —Write-Ahead Logging) [p.51-52]
8. 事务故障恢复: 反向扫描 + UNDO (逆操作写入 BI) [p.55-56]
9. 系统故障恢复: 正向扫描分队列 → UNDO(反向) + REDO(正向) [p.58-59]
10. 检查点维护日志文件 4 步骤 [p.72]
11. 检查点恢复: 五种事务判别 (Tc 前后 / Tf 前后) [p.75]
12. 利用检查点的恢复 5 步骤 [p.76-78]
13. 数据库镜像: 自动复制 + 一致性保证 [p.80]
14. 三类故障恢复方法总结: UNDO / UNDO+REDO / 重装备份 +REDO [p.85]